

Polyhedra Navigation for Macro Placement

A Novel Approach to the Partcl/HRT Macro Placement Challenge 2026

Part I: Problem Understanding

1.1 The Competition

The Partcl/HRT Macro Placement Challenge 2026 offers \$49K+ in prizes for better macro placement algorithms. Deadline: May 21, 2026.

Evaluation hardware: AMD EPYC 9655P (16 cores, 100GB RAM) + NVIDIA RTX 6000 Ada 48GB. Hard timeout: 1 hour per benchmark.

Benchmarks: 17 ICCAD04 IBM benchmarks (ibm01 through ibm18), with 246-537 hard macros, 7K-16K nets, and 43-53% area utilization. Tier 2 evaluation uses NG45 commercial designs through the full OpenROAD flow.

Proxy cost: $1.0 \times \text{Wirelength} + 0.5 \times \text{Density} + 0.5 \times \text{Congestion}$. Zero hard macro overlaps required.

Current leaderboard:

Rank	Entry	Avg Proxy Cost
1	RePIAce (baseline)	1.4578
2	Will (Partcl)	1.5338
3	SA (baseline)	2.1251

RePIAce consistently outperforms SA by 15-55% across all benchmarks.

1.2 The Formal Problem

Inputs:

- Canvas: bounded rectangle $[0, W] \times [0, H]$

- N rectangular macros with fixed dimensions (w_i, h_i), classified as hard or soft, movable or fixed
- Netlist hypergraph: M nets, each connecting a subset of macros
- Density grid: R rows x C columns

Decision variables: Center positions $p_i = (x_i, y_i)$ for each movable hard macro. Continuous, no grid snapping. Effective degrees of freedom: $2 \times |\text{movable hard macros}|$, typically 400-1000 real-valued variables.

Objective: Minimize $f(p) = WL(p) + 0.5 * D(p) + 0.5 * C(p)$ where:

- WL = sum of half-perimeter wirelength across all nets (convex, piecewise linear)
- D = mean of top 10% grid cell density values (non-convex)
- C = smoothed mean of top 5% congestion ratios (non-convex, piecewise constant)

Hard constraints:

- Canvas bounds: $4N$ linear inequalities (trivial)
- Non-overlap for hard macros: for every pair (i,k) , at least one of four linear separations must hold (the sole source of NP-hardness)
- Fixed macros: prescribed positions unchanged

1.3 The Decomposition Theorem

Given a fixed assignment of all disjunctive constraints (choosing one of the four separation conditions for each pair), the remaining problem is:

```

minimize WL(p)           (convex, piecewise linear)
subject to p in P_sigma (intersection of half-spaces = convex polyhedron)

```

This is a linear program, solvable in polynomial time. The entire computational difficulty resides in selecting the disjunctive assignments.

1.4 What a Polyhedron Is and Why

Take two macros i and k . Assigning "i left of k" yields:

$$x_i + w_i/2 \leq x_k - w_k/2$$

This is a linear inequality – a half-space. For every macro pair, one of four such inequalities is chosen. Combined with canvas bounds (also linear), the set of all valid placement

coordinates satisfying every inequality simultaneously is the intersection of half-spaces. By definition, this is a convex polyhedron. This is not a modeling choice; it is the only shape that arises from purely linear inequalities.

Inside any single polyhedron, the HPWL objective is convex, so there is a single global optimum findable in polynomial time. All the NP-hardness is in choosing which polyhedron to be in.

Part II: Why Current Methods Are Suboptimal

2.1 The Standard Pipeline

Stage 1 (Global placement): Remove the non-overlap constraint. Replace with smooth density penalty (electrostatic analogy). Solve with gradient descent. Macros overlap heavily.

Stage 2 (Legalization): Greedily resolve overlaps (Tetris-style). One-shot, irreversible commitment to a specific polyhedron. No optimality guarantee.

Stage 3 (Detailed placement): Local refinement within the chosen polyhedron (small shifts) and between neighboring polyhedra (swaps), guided by greedy heuristics.

Critical weakness: Information destruction at the legalization boundary. Stage 1 operates in a fantasy world where overlaps are allowed. Stage 2 makes a one-shot irreversible commitment. Stage 3 can never say “we should have legalized differently.” There is no backward channel from legalization to global placement.

2.2 What Our Approach Changes

- **No relaxation.** Every assignment defines a polyhedron where all macros are non-overlapping by construction. Always in feasible space.
 - **Discrete and continuous problems communicate.** LP dual variables tell the navigation layer which pairwise relations are expensive, directly informing the next move.
 - **Polyhedron choice is revisable.** The search can move between polyhedra freely, exploring thousands of fundamentally different non-overlapping arrangements.
 - **LP is solved exactly within each polyhedron.** Not a heuristic, not an approximation – the optimal continuous placement for the given topology.
-

Part III: The Search Space

3.1 Raw Size

For n_h hard macros, the number of possible disjunctive assignments is $4^{\{n_h(n_h-1)/2\}}$.

The n_h choose 2 count comes from enumerating all unordered pairs: macro 1 vs 2, macro 1 vs 3, ..., macro 1 vs n_h , macro 2 vs 3, ..., macro ($n_h - 1$) vs n_h . We do not count (1,2) and (2,1) separately because "1 left of 2" and "2 right of 1" are the same constraint.

Benchmark	n_h	Pairs	Search space	log10
ibm01	246	30,135	$4^{\{30,135\}}$	~18,127
ibm09	369	67,896	$4^{\{67,896\}}$	~40,847
ibm18	537	143,916	$4^{\{143,916\}}$	~86,590

3.2 Why This Is Still Tractable

The search space is absurd for enumeration. But:

- **We never start cold.** We start from a known good placement (RePIAce baseline), extract its assignment, and explore the neighborhood.
- **The neighborhood is searchable.** Single-flip neighbors of an assignment number ~90,000 for ibm01. Most single-flip neighbors of a feasible assignment are themselves feasible.
- **We only need local improvement.** Even 5% improvement over RePIAce likely wins. We explore thousands to millions of polyhedra in the local neighborhood, not the entire space.

Part IV: The Hierarchical Framework

4.1 The Core Idea – Clusters of Polyhedra

Existing solutions find a decent polyhedron in some cluster of polyhedra. What if we could find a better cluster?

Partition macros into k groups. Consider only inter-group pairwise assignments. This reduces the search from $4^{\{30,000\}}$ (for ibm01) to something tractable:

k (groups)	Coarse pairs	Coarse search space
5	10	$4^{\{10\}} \sim 10^6$

10	45	$4^{45} \sim 10^{27}$
15	105	$4^{105} \sim 10^{63}$

Each point in the coarse space defines a cluster of polyhedra: all fine-grained assignments consistent with the inter-group relations.

4.2 Mathematical Formulation

Coarse assignment: A function τ from group pairs to $\{L, R, A, B\}$, assigning a collective relative position to each pair of macro groups. $\tau(a,b) = L$ means every macro in G_a is to the left of every macro in G_b .

Cluster: $C(\tau) = \{ \sigma \text{ in } \Sigma_{\text{feas}} : \sigma \text{ consistent with } \tau \}$. The set of all fine-grained feasible assignments whose inter-group pairs match τ .

Cluster cost: $F(\tau) = \min \text{ over } \sigma \text{ in } C(\tau) \text{ of } [\min \text{ over } p \text{ in } P_{\sigma} \text{ of } f(p)]$. The best achievable proxy cost among all placements consistent with coarse assignment τ .

Quotient space: $\Sigma_{\text{feas}} / \sim$ where $\sigma_1 \sim \sigma_2$ iff they agree on all inter-group pairs. The original optimization factors as: $\min \text{ over } \tau \text{ of } F(\tau)$.

This is a Benders decomposition: inter-group variables are the “master” problem, intra-group variables are the “subproblem,” and dual information from the subproblem guides the master search.

4.3 Three-Level Architecture

Level 1 (Coarse Topology): Search over inter-group assignments τ in $\{L,R,A,B\}^{k(k-1)/2}$. Evaluated by running Level 2 as a subroutine. Methods: SA, parallel tempering, Bayesian optimization on the coarse space.

Level 2 (Fine Assignment): Given τ from Level 1, optimize intra-group pairwise assignments via local search with dual-variable-guided proposals. Methods: Metropolis-Hastings with LP dual ranking, cluster moves that re-solve entire groups from scratch.

Level 3 (Continuous Solver): Given a complete assignment σ , solve the HPWL LP and optionally run short gradient-based refinement for density/congestion. Returns optimal positions and dual variables.

4.4 Grouping Strategies

Connectivity-based: Spectral clustering on the netlist hypergraph. Macros sharing many nets in the same group. Ensures intra-group wiring captures densest connectivity.

Size-based: Group by area quantile. Large macros dominate feasibility, so resolving their positions first is most impactful.

Recursive bisection: Split canvas into halves recursively. At each level, solve balanced min-cut on the netlist. Produces a binary tree of $O(n \log n)$ decisions instead of $O(n^2)$ quaternary decisions.

4.5 Dual Variables as Gradient Information

The LP dual variable d_{ik} for pair (i,k) gives the marginal cost of that constraint – how much the optimal HPWL would decrease if the constraint were relaxed. Large $|d_{ik}|$ means the separation requirement is expensive.

Proposal distribution: $P(\text{flip pair } (i,k))$ proportional to $|d_{ik}|^\alpha$, where α controls exploration vs exploitation.

Direction of flip: The dual also suggests which of the three alternatives (R, A, B when current is L) is most promising, estimable from the current LP solution without solving a new LP.

4.6 Cluster Pruning Cascade: Deciding Which Clusters Are Worth Searching

The most expensive operation in Level 1 is evaluating a coarse assignment τ by running the full Level 2 search inside the cluster $C(\tau)$. Most candidate coarse assignments are bad. The key to making Level 1 tractable is eliminating bad clusters cheaply before committing to expensive evaluation – the same principle used in exoplanet search, where stellar properties (observable from Earth) eliminate most stars before you ever analyze their planets.

Each filter below computes a relaxation of the cluster cost: a function $F_{\text{hat}}(\tau) \leq F(\tau)$ that is cheaper than $F(\tau)$ but provides a valid lower bound. If $F_{\text{hat}}(\tau) > F(\tau^*)$ where τ^* is the best known solution, the entire cluster is provably suboptimal and can be pruned. This is branch-and-bound applied to the quotient space.

Filter 1: Geometric Feasibility (cost: ~ 0)

Some coarse assignments are physically impossible regardless of the fine details. If τ implies “group A left of group B left of group C” and the minimum total width of all macros in $A + B + C$ exceeds the canvas width, the entire cluster contains only empty polyhedra. Check all chains of left-right and above-below relations implied by τ by summing minimum macro dimensions along each chain. This is analogous to filtering out brown dwarfs – instant disqualification on fundamental physical grounds.

Filter 2: Wirelength Lower Bound (cost: ~ 1 microsecond)

Without solving any LP, compute a lower bound on the wirelength achievable under τ . If τ says group A is left of group B, then every macro in A has its center to the left of every macro in B. The minimum horizontal separation between any macro i in G_a and any macro j in G_b is at least $(w_i + w_j)/2$. For any net connecting macros in both groups, this forced separation contributes a minimum wirelength. Sum these minimum contributions across all nets and all inter-group separations. If this lower bound already exceeds the best known proxy cost, the entire cluster is worthless. This is the “star is outside the habitable zone” filter – the fundamental physics makes life impossible regardless of planetary details.

Filter 3: Net Cut Cost (cost: ~10 microseconds)

Count how many nets are “cut” by each inter-group boundary. If τ places groups A and B on opposite sides of the canvas, every net connecting macros in both groups must cross that distance. The total cut cost – sum over all inter-group boundaries of (number of crossing nets) times (minimum crossing distance) – is a fast proxy for total wirelength. Coarse assignments that put heavily connected groups far apart have high cut cost. This is directly analogous to graph partitioning quality metrics and is the “star has wrong metallicity for rocky planets” filter.

Filter 4: Area Packing Feasibility (cost: ~100 microseconds)

For each spatial region implied by τ , check whether the macros assigned to that region physically fit. If τ implies groups A, B, C are all in the left half of the canvas, compute the total area of macros in those groups and compare to the area of the left half. If the packing density exceeds ~85% (a rough empirical threshold for rectangular packing), it is very unlikely that any feasible fine-grained assignment exists within that region. More expensive than Filters 1-3 because it requires reasoning about spatial implications of τ , but still far cheaper than an LP. This is the “planet is the right size but atmosphere is 100% CO₂” filter.

Filter 5: Relaxed LP Bound (cost: ~1 millisecond)

Solve a tiny LP that only includes inter-group constraints (from τ) and canvas bounds, treating each group as a single “super-macro” with aggregate dimensions. This gives k position variables instead of n_h , so the LP has maybe 20 variables instead of 500. The optimal value is a tighter lower bound than Filter 2 because it accounts for constraint interactions between groups. If this bound already exceeds the best known solution, prune. This is the “check the planet’s orbit and mass but not its atmosphere” filter.

The Full Cascade

For each candidate coarse assignment τ :

```

if fails geometric feasibility:      skip      (cost: ~0)
if wirelength lower bound > best:    skip      (cost: ~1 us)
if net cut cost > threshold:         skip      (cost: ~10 us)
if area packing infeasible:          skip      (cost: ~100 us)
if relaxed LP bound > best:          skip      (cost: ~1 ms)
else: run full Level 2 evaluation      (cost: ~1-10 s)

```

Most candidates die at Filters 1-3. Very few reach the expensive evaluation. The ratio of filtered to evaluated candidates should be 1000:1 or higher, meaning we effectively search 1000x more of the coarse space within the same time budget.

Connection to Branch-and-Bound

This cascade is a branch-and-bound scheme on the quotient space $\text{Sigma_feas} / \sim$. Each filter provides a bounding function of increasing tightness and increasing cost. The pruning is valid because every filter computes a lower bound: if the lower bound exceeds the incumbent, no solution in the cluster can improve on the incumbent. This is the same mathematical principle underlying all modern MIP solvers (CPLEX, Gurobi, HiGHS) – the novelty is applying it to the specific structure of the macro placement quotient space.

Connection to Multi-Fidelity Search

The cascade is also an instance of multi-fidelity optimization. Low-fidelity evaluations (Filters 1-5) are cheap but noisy. High-fidelity evaluation (Level 2) is expensive but accurate. The optimal strategy is to use low-fidelity evaluations to identify a small candidate set, then allocate the expensive high-fidelity budget to that set. This is standard in Bayesian optimization and hyperparameter tuning, but has not been applied to placement topology search.

Empirical Calibration Needed

The thresholds for Filters 3-4 require calibration on actual benchmark data. Run the cascade on a large sample of random coarse assignments for ibm01, measure the true Level 2 cost for a subset, and determine the false-negative rate of each filter (how often does a filter prune a cluster that actually contained a good solution). A filter with >1% false-negative rate needs its threshold loosened. The filters should be conservative – it is better to evaluate a few unnecessary clusters than to prune the one that contained the global optimum.

Part V: Relationship to MCMC and Advanced Navigation

5.1 The MCMC Connection

The polyhedra search is a Markov chain on the space of feasible disjunctive assignments. Define the "energy" $E(\sigma) = \text{LP-optimal cost for assignment } \sigma$. The Metropolis-Hastings version: propose a neighbor σ' (flip one pair), solve the LP, accept with probability $\min(1, \exp(-\beta * (E(\sigma') - E(\sigma))))$. The stationary distribution concentrates on low-cost polyhedra as $\beta \rightarrow \infty$.

5.2 Why Naive MCMC Fails

- Most random assignments are infeasible (empty polyhedra)
- The graph has exponential diameter in the worst case
- Mixing time may be exponential if the feasible region has bottlenecks

5.3 Advanced MCMC Methods

Parallel tempering (replica exchange): Run K chains at different temperatures. Hot chains explore freely, crossing barriers. Cold chains exploit. Periodically swap configurations between adjacent temperatures.

Wang-Landau / flat histogram: Estimate the density of states $g(E) = |\{\sigma : E(\sigma) \sim E\}|$, driving the chain into unexplored energy regions. Reveals the landscape structure.

Population annealing: Maintain a population of N configurations. Resample with Boltzmann weights at each temperature step, then apply MCMC moves. Combines multi-start exploration with annealing selection pressure.

Cluster moves: Instead of flipping one pair, simultaneously re-assign all pairs involving a single macro. This changes $O(n_h)$ pairwise relations at once, analogous to Swendsen-Wang cluster algorithms in statistical physics.

5.4 Frontier Techniques Not Yet Applied

Discrete Schrodinger bridges: Define a bridge between a "hot" distribution (uniform over feasible assignments) and a "cold" distribution (concentrated on low-cost assignments). The bridge gives the optimal stochastic process for the transition – a time-varying drift that efficiently funnels probability mass toward good solutions. Strictly more powerful than SA because it optimizes the entire trajectory, not just the instantaneous transition. Key papers: Discrete Adjoint SB Sampler (Feb 2026), DDSBM, Categorical SBM (ICML 2025). Note: the bridge does not directly minimize placement cost. It solves a transport problem. The placement cost defines the target distribution. You still need LP solvers for evaluation.

Algorithmic probability sampling (Kolmogorov complexity): Dingle and Hutter

(DeepMind, Feb 2026) showed optima of combinatorial problems disproportionately have low Kolmogorov complexity. Bias sampling toward compressible assignments – ones describable by simple rules. Practical version: use compressed length as an energy bonus in the MCMC acceptance probability.

LLM-guided evolutionary search (AlphaEvolve): Use an LLM to evolve the heuristic function guiding the MCMC search – the proposal distribution, temperature schedule, cluster move selection. Rather than hand-designing these, let the LLM generate candidate code, evaluate on benchmarks, and evolve.

Neural surrogate evaluation: Train a model that maps assignment sigma to approximate LP cost, bypassing the LP solve during exploration. Use for MCMC accept/reject, validate with true LP only for top candidates. Multi-fidelity cascade: hash lookup ($O(1)$) $\rightarrow$ neural surrogate ($O(\text{ms})$) $\rightarrow$ warm-started LP with early termination ($O(10\text{ms})$) $\rightarrow$ full LP ($O(100\text{ms})$).

Part VI: Research Landscape

6.1 Cross-Disciplinary Theories (Not Yet Applied to Placement)

Tropical geometry: HPWL is literally a tropical polynomial in the $(\max, +)$ semiring. Tropical gradient descent (Talbut & Monod, 2024) achieves convergence matching classical GD on tropically convex problems. Tropical attention (Hashemi et al., 2025) builds neural architectures preserving polyhedral decision boundaries.

Cavity method / survey propagation: The assignment space maps to a factor graph with 4-state Potts-like spins. Survey propagation could identify clusters of assignments, characterize their hierarchy, and guide search toward under-constrained variables.

Complexification: Extending coordinates from $\mathbb{R}^{2N}$ to $\mathbb{C}^{2N}$ makes the complement of the constraint hyperplanes path-connected. Homotopy continuation can trace paths between optima of different polyhedra through complex space.

Optimal transport: Density minimization is fundamentally an OT problem. Entropy-regularized OT via Sinkhorn iterations provides a differentiable density-spreading mechanism. The Benamou-Brenier formulation interprets placement as a Wasserstein gradient flow.

Stratified Morse theory: The feasible region (union of polyhedra with shared faces) is a stratified space. Morse inequalities bound local minima count by Betti numbers of the feasible region. Persistent homology of the objective filtration reveals basin structure.

6.2 What Exists in EDA Literature

- Sequence pair representations with SA (standard, well-refined over 30 years)
- Analytical placement with electrostatic density penalty (ePlace/DREAMPlace/RePIAce)
- RL-based constructive placement (Google Nature paper, various follow-ups)
- Constraint graph representations with greedy legalization
- One paper (2024) combining RL for placement order + LP for wirelength within fixed relative positions – closest to our Level 3

6.3 What Does Not Exist

No existing work does any of the following:

- Explicitly formulates placement as navigation over a graph of convex polyhedra
- Uses LP dual variables to guide discrete navigation between polyhedra
- Defines clusters of polyhedra via macro grouping with a quotient space formulation
- Implements a three-level (coarse topology / fine assignment / continuous LP) architecture
- Applies warm-started LP evaluation for single-constraint-flip navigation in placement

The composition is novel. Individual pieces (LP solving, local search, spectral clustering) are well-understood. The innovation is the architecture and its connection to the decomposition theorem.

Part VII: The Full Stack – Five Modules

Module 1: Assignment Extractor

Input: Legal placement tensor $[N, 2]$, macro sizes $[N, 2]$. **Output:** Dictionary mapping each hard macro pair (i,k) to $\{L, R, A, B\}$.

Logic for each pair (i,k) :

```
if x_i + w_i/2 <= x_k - w_k/2:    L   (i left of k)
elif x_k + w_k/2 <= x_i - w_i/2:  R   (k left of i)
elif y_i + h_i/2 <= y_k - h_k/2:  B   (i below k)
elif y_k + h_k/2 <= y_i - h_i/2:  A   (i above k)
else: ERROR (overlap)
```

Note: when multiple conditions hold (macros separated in both x and y), tie-breaking affects neighborhood structure. Record all valid directions.

Test: Extract assignment from greedy row placer output. Verify every hard macro pair has at least one valid separation. Count pairs with multiple valid directions.

Module 2: LP Solver

Input: Complete assignment sigma, macro sizes, canvas dimensions, net connectivity.

Output: Optimal positions, optimal HPWL, dual variables for all pairwise constraints.

LP formulation: introduce bounding-box variables for each net j:

```
min sum_j (u_j^max - u_j^min + v_j^max - v_j^min)

s.t. u_j^max >= x_i      for all i in S_j, for all j
     u_j^min <= x_i      for all i in S_j, for all j
     [same for y]
     [half-space constraints from sigma]
     [canvas bounds]
```

Size for ibm01: ~30,000 variables, ~90,000 constraints. Solvable in 1-10ms (HiGHS).

Warm-starting: when sigma' differs by one pair flip, only 1-2 constraints change. Dual simplex re-optimizes in O(n) pivots. Sub-millisecond.

Critical test: Extract assignment from RePIAce output, solve LP, compare LP-optimal HPWL to RePIAce's HPWL. This answers the most important open question: how much continuous slack exists within RePIAce's polyhedron.

Module 3: Neighbor Generator

Input: Current assignment sigma, dual variables from Module 2. **Output:** Ranked list of candidate neighbor assignments.

Uses dual variable magnitude to rank proposals: $P(\text{flip pair } (i,k))$ proportional to $|d_{\{i,k\}}|^\alpha$.

For each proposed flip, checks feasibility by attempting warm-started LP solve.

Test: From RePIAce assignment, generate all single-flip neighbors for top-50 highest-dual pairs. Report feasibility fraction and cost distribution.

Module 4: Navigator

Input: Initial assignment, time budget. **Output:** Best assignment found, its optimal

placement.

The search loop: propose (Module 3), evaluate (Module 2), accept/reject. Pluggable strategies:

- Greedy descent (accept only improvements)
- Metropolis-Hastings (accept worse with temperature-dependent probability)
- Parallel chains across 16 cores

Test: From RePIAce assignment on ibm01, run 1000 iterations of greedy descent. Plot cost vs. iteration. Measure seconds per iteration.

Module 5: Hierarchical Grouper

Input: Netlist hypergraph, macro properties. **Output:** Macro groups, coarse assignment extraction.

Spectral clustering on net adjacency matrix. Provides Level 1 search layer.

Test: Cluster ibm01's 246 macros into 10 groups. Report group sizes, inter/intra-group net ratios, and whether coarse assignment from RePIAce shows clean majority votes.

Part VIII: Implementation Plan

Phase 1: Proof of Concept (Week 1)

Build Modules 1 + 2 only. This answers the most critical question (continuous slack) and produces a working submission.

Submission code: Extract assignment from RePIAce output -> solve LP -> return LP-optimal positions. Use TILOS evaluator for density/congestion.

Key deliverable: Quantified gap between LP-optimal HPWL and RePIAce HPWL within the same polyhedron. If significant (>5%), this is already a contribution.

Phase 2: Flat Navigation (Week 2)

Add Modules 3 + 4. Start from LP-optimal solution within RePIAce's polyhedron. Search neighboring polyhedra using dual-guided proposals. Each accepted move triggers warm-started LP re-solve.

Key deliverable: Cost-vs-iteration curves showing whether better polyhedra exist nearby.

Phase 3: Hierarchical Search (Week 3)

Add Module 5. Only worth building if Phase 2 shows (a) better polyhedra exist but (b) flat search gets stuck. The hierarchy enables non-local jumps between basins.

Implement and calibrate the cluster pruning cascade (Section 4.6). Run the five filters on a large sample of random coarse assignments for ibm01. Measure: (a) what fraction of candidates each filter eliminates, (b) the false-negative rate of each filter, (c) the effective speedup ratio (candidates evaluated vs. candidates searched). Tune filter thresholds so the false-negative rate is below 1% for each filter.

Key deliverable: Comparison of flat vs. hierarchical search quality within same time budget. Pruning cascade calibration data showing filter elimination rates and false-negative rates.

Phase 4: Tuning and Robustness (Week 4)

Run across all 17 benchmarks. Calibrate temperature schedules, proposal distributions, time budgets. Handle edge cases.

Key deliverable: Final submission achieving lower proxy cost than RePIAce across all benchmarks.

Part IX: Computational Budget

Time Allocation (3,600 seconds total)

Phase	Time	Activity
0 (Init)	60s	Extract baseline assignment, set up LP, calibrate
1 (Coarse search)	1,200s	Cascade filters ~100K candidates, full-evaluates ~100-200 survivors (parallelized x16)
2 (Fine refinement)	1,800s	Deep Level 2 run on best tau*
3 (Polish)	540s	Gradient-based continuous refinement

Per-Operation Cost Estimates

Operation	Time	Notes

Cascade Filters 1-3	0.001-0.01 ms	Per candidate, eliminates ~99%
Cascade Filters 4-5	0.1-1 ms	Per surviving candidate
LP solve (cold)	5-50 ms	Depends on n_h
LP solve (warm)	0.1-1 ms	Single constraint change
TILOS evaluation	10-50 ms	Full proxy cost
Level 2 iteration	1-5 ms	Warm LP + accept/reject
Level 2 full run (per tau)	1-10 s	1,000-5,000 iterations

Part X: Risk Mitigation

Risk 1: LP placement has terrible density/congestion

The LP optimizes only HPWL. Macros may cluster pathologically.

Mitigation: After LP solve, run 100-200 steps of projected gradient descent adding a density penalty while staying within the polyhedron. Cheap (milliseconds) and handles density/congestion without changing the discrete assignment.

Risk 2: Most neighbor polyhedra are infeasible

If feasibility fraction is very low ($<1\%$), the search space is too sparse for local navigation.

Mitigation: Use constraint propagation to pre-filter proposals. If flipping pair (i,k) from L to R, check transitivity constraints with all macros adjacent to both i and k before attempting the LP. Also: larger moves (flipping 2-3 pairs simultaneously chosen to be geometrically consistent) may have higher feasibility rates.

Risk 3: Continuous slack is negligible

If the LP within RePIAce's polyhedron gives nearly identical HPWL, then Module 2 alone does not help.

Mitigation: This means all improvement must come from finding better polyhedra (Modules 3-5). Phase 1 still provides the machinery (LP solver, dual variables) needed for the navigation phases. The LP is never wasted – it provides exact evaluation and gradient information for the discrete search.

Risk 4: LP is too slow at scale

For ibm18 (537 macros, ~144K pairs), the LP has ~290K variables and ~870K constraints.

Mitigation: (a) Sparse LP formulations exploiting net structure. (b) Only include pairwise constraints for macro pairs that are geometrically close (distant macros are trivially separated). (c) Iterative refinement: solve a reduced LP with nearby-pair constraints, check for violations among distant pairs, add violated constraints, re-solve.

Part XI: Open Questions Requiring Empirical Answers

1. **Continuous slack magnitude.** How much better is the LP-optimal HPWL within RePIAce's polyhedron vs. RePIAce's actual HPWL? This is question zero. Everything else depends on it.
2. **Feasibility fraction of neighbors.** What fraction of single-flip neighbors of a good assignment are themselves feasible? If >10%, local search is viable. If <0.1%, we need non-local moves.
3. **Dual variable informativeness.** Do high-dual pairs actually produce the best improving flips? Validate by comparing dual-ranked proposals to random proposals.
4. **Density/congestion behavior under LP placement.** Does the LP-optimal placement within a given polyhedron produce reasonable density and congestion, or pathological clustering?
5. **Grouping cleanliness.** When extracting coarse assignments from RePIAce, are the majority votes decisive (>80% agreement) or mixed? Mixed votes suggest the grouping is wrong or the coarse abstraction is too lossy.
6. **LP warm-start speedup.** What is the actual ratio of cold-start to warm-start LP solve time on the ICCAD04 benchmarks?
7. **Scaling behavior.** How does the cost-per-iteration and improvement-per-iteration scale from ibm01 (246 macros) to ibm18 (537 macros)?
8. **Cascade filter false-negative rate.** When the pruning cascade (Filters 1-5 in Section 4.6) rejects a coarse assignment, how often does it incorrectly reject a cluster that actually contained a solution better than the incumbent? Run the cascade on a large sample of random coarse assignments for ibm01, evaluate a subset with the full Level 2 solver, and measure how many good clusters each filter incorrectly prunes. A filter with >1% false-negative rate needs its threshold loosened.
9. **Filter tightness vs. cost trade-off.** What fraction of candidates does each filter eliminate? If Filter 2 (wirelength lower bound) already kills 99% of candidates, Filters 3-5

add little value and can be skipped to simplify the implementation. Conversely, if Filter 2 only kills 50%, the tighter (and more expensive) filters are essential.

Part XII: Why This Could Win

The Innovation Prize Argument

No existing tool explicitly decomposes placement into “which polyhedron” (NP-hard) and “where within the polyhedron” (LP-solvable). This is a fundamentally new architectural choice for placement, connecting the EDA problem to 50 years of mathematical programming theory (Benders decomposition, disjunctive programming, LP duality). The innovation prize specifically rewards creative and technically innovative approaches.

The Proxy Cost Argument

RePIAcE uses a smooth relaxation that never solves the non-overlap problem exactly within any topology. Our approach solves HPWL exactly within each topology (via LP) and uses the dual information to navigate between topologies intelligently. Even if our discrete search only visits 10,000 polyhedra out of $10^{18,000}$, those 10,000 are chosen using the LP’s exact gradient information rather than the heuristic gradient of a relaxed objective.

The Composability Argument

The framework is modular. Level 1 navigation can be upgraded independently (from greedy to parallel tempering to Schrodinger bridges) without changing Levels 2-3. The LP solver can be swapped (HiGHS to Gurobi). The grouping strategy can be changed. Each upgrade compounds with the others. This gives us a development path where each week’s work strictly improves on the previous week.

Appendix A: Competition API Summary

Benchmark object fields: canvas_width, canvas_height, num_macros, num_hard_macros, num_soft_macros, macro_positions [N,2], macro_sizes [N,2], macro_fixed [N], num_nets, grid_rows, grid_cols, hard_macro_indices, soft_macro_indices.

Placer interface: Takes a Benchmark, returns [num_macros, 2] tensor of (x,y) center positions. Only hard macro positions optimized; soft macros stay at initial positions.

Evaluation: compute_proxy_cost(placement, benchmark, plc) returns dict with proxy_cost, wirelength_cost, density_cost, congestion_cost, overlap_count.

Validation: `validate_placement` checks shape, NaN/Inf, canvas bounds, fixed positions, zero overlaps.

Net connectivity: Stored in `PlacementCost` object (`plc`), not in Benchmark tensors. Accessible via `plc.modules_w_pins` and TILOS API.

Appendix B: Key References

- Decomposition theorem and formal problem statement: provided problem PDF
- Tropical gradient descent: Talbut & Monod, arXiv:2405.19551 (2024)
- Kolmogorov complexity and optimization: Dingle & Hutter, Entropy 28(2):226 (2026)
- Discrete Schrodinger bridges: arXiv:2602.08243 (2026)
- Chip placement with diffusion: Lee et al., ICML 2025, arXiv:2407.12282
- Fisher-Rao gradient flows for LPs: Muller & Montufar, arXiv:2403.19448 (2024)
- AlphaEvolve: DeepMind, 2025
- Disjunctive programming: Balas (2011), Wiley Encyclopedia
- P-split formulation hierarchy: Kronqvist et al., Mathematical Programming (2025)
- Learning placement order with LP: ScienceDirect (2024)
- TILOS MacroPlacement: github.com/TILOS-AI-Institute/MacroPlacement
- Competition repo: github.com/partcleda/macro-place-challenge-2026